

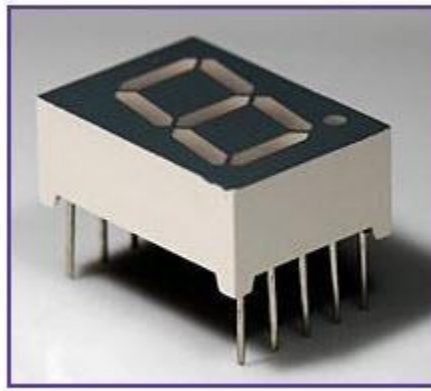
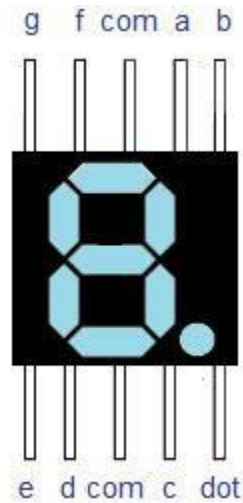
Internet of Things

Practical - 7

Aim : To interface Seven Segment Display with Raspberry Pi.

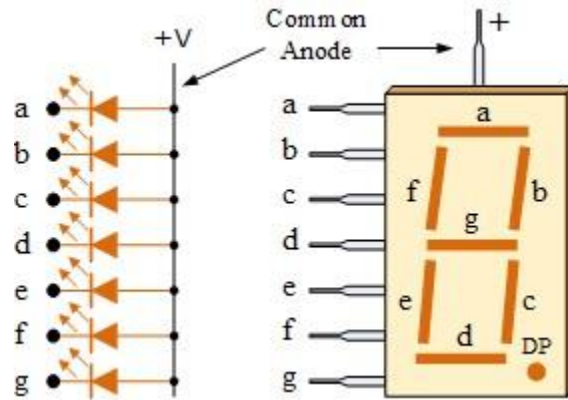
Theory :

A 7-segment display is an electronic component made of seven individual light-emitting diode (LED) segments. Arranged in a figure-eight shape, they light up in different combinations to show numbers from 0 to 9 and basic letters.



The seven segment display are of two types : -

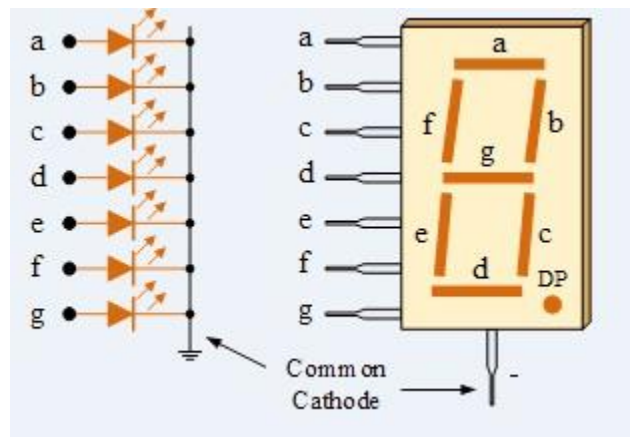
1. **Common Anode :-** Common anode 7-segment display, all the positive pins (anodes) of the 7 individual LED segments are tied together to a single common pin. This common pin connects to a positive voltage supply (V_{cc}), while the individual negative pins (cathodes) control each segment (labeled A through G).



To light an LED, you can apply a low or ground to its specific cathode.

2. Common Cathode :-

In a common cathode seven-segment display circuit, all the negative pins (cathodes) of the 7 individual LED segments (A–G) and the decimal point are joined together and connected directly to the ground (0V). Individual positive pins (anodes) receive a HIGH logic signal (3.3V or 5V) through current-limiting resistors to turn on specific segments



TM1637

TM1637 is a 4-digit common cathode type 7-segment LED display module controlled by a dedicated Titan Micro Electronics driver IC. It uses a 2-wire serial interface (CLK and DIO) alongside power pins (VCC and GND), reducing a typical 12-pin display requirement down to just 4 wires. Operating at 3.3V to 5.5V, it features 8 software-adjustable brightness levels and a central colon ideal for clocks.



Part – I : Connection

Connect the Internet cable to Raspberry Pi kit or switch on the WiFi. Connect the Seven Segment Display to Raspberry pi by doing the following connections:

**CLK (Clock) : Pin no. 16 DIO(Data I/O) : Pin no. 18 VCC (Power Supply) :
Pin no. 4 GND (Ground) : Pin no. 14**

Part – II : Program for Library installation (Save in Documents Folder in Raspberry Pi)

```
#tm1637.py
```

```
# https://raspberrytips.nl/tm1637-4-digit-led-display-raspberry-pi/
```

```
import sys
import os
import time
import
RPi.GPIO
as IO
```

```
IO.setwarnings(False)
```

```
IO.setmode(IO.BCM)
```

```
HexDigits =
```

```
[0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71]
```

```
ADDR_AUTO = 0x40
```

```
ADDR_FIXED = 0x44
```

```
STARTADDR = 0xC0
```

```
BRIGHT_DARKEST = 0
```

```
BRIGHT_TYPICAL = 2
```

```
BRIGHT_HIGHEST = 7
```

```
OUTPUT = IO.OUT
```

```
INPUT = IO.IN
```

```
LOW = IO.LOW
```

```
HIGH = IO.HIGH
```

```
class TM1637:
```

```
    __doublePoint = False
```

```
    __Clkpin = 0
```

```
    __Datapin = 0
```

```
    __brihtnes = BRIGHT_TYPICAL;
```

```
    __currentData = [0,0,0,0];
```

```
    def __init__( self, pinClock, pinData, brightnes ):  
self.__Clkpin = pinClock      self.__Datapin =  
pinData      self.__brihtnes = brightnes;  
        IO.setup(self.__Clkpin,OUTPUT)  
        IO.setup(self.__Datapin,OUTPUT)  
    # end __init__
```

```
    def Clear(self):  
        b      = self.__brihtnes;  
point      = self.__doublePoint;  
self.__brihtnes = 0;  
        self.__doublePoint = False;  
data = [0x7F,0x7F,0x7F,0x7F];  
        self.Show(data);  
        self.__brihtnes = b;          # restore saved brightnes  
self.__doublePoint = point;  
    # end Clear  
    def ShowInt(self, i):      s  
= str(i)      self.Clear()  
    for i in range(0,len(s)):
```

```

        self.Show1(i, int(s[i]))

    def Show( self, data ):          for i in
range(0,4):          self.__currentData[i]
= data[i];

        self.start();
        self.writeByte(ADDR_AUTO);
        self.stop();
        self.start();
        self.writeByte(STARTADDR);
for i in range(0,4):
self.writeByte(self.coding(data[i]));
        self.stop();
        self.start();
        self.writeByte(0x88 + self.__brihtnes);
        self.stop();
# end Show

    def SetBrihtnes(self, brihtnes):  # brihtnes 0...7
if( brihtnes > 7 ):          brihtnes = 7;      elif(
brihtnes < 0 ):          brihtnes = 0;

        if( self.__brihtnes != brihtnes):
self.__brihtnes = brihtnes;
self.Show(self.__currentData);
        # end if
# end SetBrihtnes

    def ShowDoublepoint(self, on):      # shows or hides the doublepoint
if( self.__doublePoint != on):          self.__doublePoint = on;
        self.Show(self.__currentData);
        # end if
# end ShowDoublepoint

    def writeByte( self, data ):
for i in range(0,8):
        IO.output( self.__Clkpin, LOW)
        if(data & 0x01):
            IO.output( self.__Datapin, HIGH)
        else:
            IO.output( self.__Datapin, LOW)
        data = data >> 1
        IO.output( self.__Clkpin, HIGH)

```

```

#endfor

# wait for ACK
IO.output( self.__Clkpin, LOW)
IO.output( self.__Datapin, HIGH)
IO.output( self.__Clkpin, HIGH)
IO.setup(self.__Datapin, INPUT)

while(IO.input(self.__Datapin)):
    time.sleep(0.001)    if(
IO.input(self.__Datapin)):
IO.setup(self.__Datapin, OUTPUT)
    IO.output( self.__Datapin, LOW)
    IO.setup(self.__Datapin, INPUT)
    #endif
# endwhile
IO.setup(self.__Datapin, OUTPUT)
# end writeByte

def start(self):
    IO.output( self.__Clkpin, HIGH) # send start signal to TM1637
    IO.output( self.__Datapin, HIGH)
    IO.output( self.__Datapin, LOW)
    IO.output( self.__Clkpin, LOW)
# end start

def stop(self):
    IO.output( self.__Clkpin, LOW)
    IO.output( self.__Datapin, LOW)
    IO.output( self.__Clkpin, HIGH)
    IO.output( self.__Datapin, HIGH)
# end stop

def coding(self, data):
if( self.__doublePoint ):
pointData = 0x80    else:
    pointData = 0;
    if(data == 0x7F):
        data = 0
    else:
        data = HexDigits[data] + pointData;
    return data
# end coding

```

```
# end class TM1637
```

Part – III : Enter the code in Thonny

#Clock.py

```
import sys import time
import os import datetime
import RPi.GPIO as GPIO
import tm1637

#CLK->GPIO23(pin 16)
#Di0->GPIO24(pin 18)

Display=tm1637.TM1637(23,24,tm1637.BRIGHT_TYPICAL)
Display.Clear()
Display.SetBrightness(1)

while(True):
    now=datetime.datetime.now()
    hour=now.hour    minute=now.minute
    second=now.second
    currenttime=[int(hour/10), hour%10 ,int(minute/10),minute%10]

    Display.Show(currenttime)
    Display.ShowDoublepoint(second % 2)

    time.sleep(1)
```

Output :-

Make a folder and keep two codes files in it.

First run the tm1637.py (library file) in Thonny and do not stop it, Now run the Clock.py (code file) in Thonny and do not stop it.

We will get the output.

